

Laboratorio 1 · notas del docente

Bisección con un agente de RL · 90 minutos

Resultados medidos el 22 de septiembre de 2026 en un MacBook con Python 3.13, semilla 1. Todo se reproduce con `docs/curso/lab01/experimentos.py`.

1 Tiempos

comando	duración
<code>python -m bisectrl</code> (con animación, 3000+3000 gen)	1-2 min; con <code>--speed 3</code> unos 40 s
<code>python -m bisectrl --no-tui</code>	0,35 s
<code>experimentos.py generaciones</code>	1 s
<code>experimentos.py recompensa</code>	1 s
<code>experimentos.py sin_medio / grafo</code>	< 1 s

La animación necesita una terminal de 120×40 . Si los monitores del laboratorio son pequeños, proyectar una sola corrida y que las parejas trabajen con `--no-tui` y el informe.

2 Plan de la sesión

min	bloque	modo
0-30	terminal desde cero: abrir, <code>cd</code> , <code>ls</code> , descargar ZIP, <code>venv</code> , <code>pip install</code> . Ir puesto por puesto. Meta: todos corren <code>--no-tui</code> y ven un informe	guiado
30-40	RL en la pizarra con la bisección como único ejemplo: estado, acción, recompensa, política. Preguntas (a)-(c) en papel	expositivo
40-55	parte 2: correr con animación, anotar hitos, leer la síntesis	parejas
55-70	parte 3: romper la recompensa; plenaria breve sobre (f)	parejas + plenaria
70-85	parte 4: dibujar el grafo, experimento <code>grafo</code> , plenaria sobre (n)	parejas + plenaria
85-90	cierre y entrega	expositivo

Las dos ideas de la sesión son (f) y (n). Si se va el tiempo, sacrificar `sin_medio` y la comparación de semillas, nunca esas dos plenarias.

Riesgo principal: la instalación

Probar el día anterior en un PC del laboratorio que `python --version` funciona, que hay internet para `pip`, y que Windows Terminal se abre. Si `pip` no tiene red, llevar un USB con `rich` y `pytest` descargados (`pip download rich pytest`) e instalar con `--no-index --find-links`.

3 Resultados esperados y respuestas

- (a) Con $\lambda = 0,5$ el ancho se divide por 2: $-1 + \log_2 2 = 0$ puntos por paso, exacto. Con $\lambda = 0,1$, el 10 % de las veces (aproximadamente, si la raíz cae uniforme) el progreso es $\log_2 10 \approx 3,3$ y el 90 % es $\log_2(1/0,9) \approx 0,15$; en promedio unos 0,47 bits, o sea $Q(0,1) \approx -0,53$. Medido: $-0,55$ a $-0,76$ según semilla. La raíz no cae uniforme, así que las cifras varían.
- (b) Sí: los tres signos bastan. Es el punto que conecta con la demostración: el invariante

$f(a_n) f(b_n) \leq 0$ es exactamente lo que la regla de conservación mantiene.

- (c) $\lambda = 1/2$. Argumento minimax: el peor caso de cualquier $\lambda \neq 1/2$ es peor que el de $1/2$. Los mejores estudiantes notarán que el promedio también lo favorece si la raíz está distribuida simétricamente.

Hitos con semilla 1 (3000+3000 generaciones):

hito	generación
primera convergencia	7
$\lambda = 1/2$ estable (300 gen seguidas)	345
invariante de Bolzano	358
$\geq 95\%$ de convergencia	391
primera demostración completa	256 (fase 2)
política voraz produce la mínima	300
primera demostración mínima durante el entrenamiento	459

Con semilla 42 y solo 200+100 generaciones (una corrida vieja en runs/) no aprende nada: es un buen ejemplo de «el informe dice honestamente que no lo logró».

- (d) $Q(\lambda)$ es el progreso medio por evaluación menos 1. $Q(0,5) = 0$ porque el progreso es exactamente 1 bit. Valores negativos = menos de 1 bit por evaluación en promedio.
- (e) Las generaciones cambian (decenas arriba o abajo); lo aprendido, no. Es el punto para distinguir *proceso estocástico* de *resultado*.
- (f) Resultado medido con «perder la raíz cuesta 0»: **0 convergencias, 2000 raíces perdidas**, éxito 0%. El agente aprende a tirar la raíz en el primer paso. Es racional: un episodio que pierde la raíz de inmediato gana $-1 + \log_2(1/(1 - \lambda))$ una sola vez y termina; bisecar 12 veces gana 0 en total (con $\lambda = 1/2$ cada paso vale exactamente 0). Terminar rápido *domina*. Es el ejemplo más limpio de *reward hacking* que se puede dar a este nivel, y conecta con la idea de que la recompensa es la especificación completa del problema.
- (g) Con paso gratis el agente sigue eligiendo $\lambda = 1/2$. La cabeza de corte usa $\gamma = 0$ y tasa $1/N$: maximiza el progreso esperado *de este paso*, y ese progreso lo maximiza el punto medio con o sin el -1 . Buen momento para decir que la suma de las recompensas de progreso telescopa: $\sum \log_2(w_{k-1}/w_k) = \log_2(w_0/w_n)$, así que el shaping no cambia qué es óptimo (Ng, Harada y Russell, 1999). No hace falta citar el artículo a los estudiantes.
- (h) Respuestas típicas buenas: «premiar por cada evaluación de f » (evaluaría infinito), «premiar que $|f(x)|$ sea pequeño» (se estanca donde f es plana), «premiar solo al final» (aprende más lento pero no se puede engañar). Cualquier respuesta que identifique *qué haría el agente* vale.
- (i) Medido sin 0,5: elige $\lambda = 0,4$ con $Q(0,4) = -0,029$, $Q(0,6) = -0,039$; la curva es simétrica en forma de «V invertida» con máximo en 0,5. Las diferencias 0,4/0,6 son ruido de muestreo; pedir que corran otra semilla para verlo.
- (j) El grafo tiene profundidad 7 ($H \rightarrow DEF \rightarrow MONO \rightarrow CONV \rightarrow SAME \rightarrow CONT \rightarrow ROOT \rightarrow QED$). Órdenes válidos: muchos; por ejemplo H1, H2, DEF, INV, WIDTH, MONO_A, MONO_B, CONV_A, CONV_B, SAME, CONT, ROOT, MID, QED. Basta con que respeten las flechas.
- (k) **D_BOLZ** es el importante: existencia de una raíz (Bolzano) no dice nada sobre a dónde converge la sucesión de puntos medios; puede haber varias raíces y hay que probar que m_n converge y que su límite es raíz. Otros: **D_LIP**, Lipschitz no está en las hipótesis y «luego es de Cauchy» no se sigue; **D_UNIQ**, falso en general; **D_TVM**, verdadero bajo derivabilidad pero irrelevante; **D_DERIV**, **D_NEWTON**, otro método; **D_DIV**, falso (acotada).

- (l) El agente aprendió un orden topológico del grafo y a evitar nodos sin salida. No «entendió» nada. Esta es la respuesta que hay que arrancarles; si dicen «el agente demostró el teorema», volver al experimento **grafo**.
- (m) ROOT dice: como $f(a_n) f(b_n) \leq 0$ para todo n y el límite es $f(c)^2$, entonces $f(c)^2 \leq 0$. Sin INV no sabemos que los productos son ≤ 0 ; el límite de $f(a_n) f(b_n)$ sigue siendo $f(c)^2$, pero nada obliga a que sea ≤ 0 . El verificador acepta la demostración de 13 pasos porque nadie le dijo que faltaba algo. Medido: el verificador devuelve `finished = True` para el orden sin INV.
- (n) El grafo es la demostración. Quien escribe el grafo demuestra; el agente lo ordena. Esto justifica el proyecto final: escribir un grafo correcto es el trabajo matemático.

(tarea)

Un distractor es «tentador» para el agente solo por el bono de exploración (UCB): lo intenta un número parecido de veces a cualquier otro nodo inválido hasta que su Q cae. Para un humano es tentador si *parece* seguirse. El agente no tiene noción de plausibilidad; el humano sí. En el informe, la tabla «saltos lógicos más frecuentes» muestra los intentos por paso.

Extras si preguntan. Estados de la regla de conservación: 3 signos con $f(x)$ posiblemente 0 dan $2 \cdot 3 \cdot 2 = 12$ combinaciones, pero con el invariante $f(a)$ y $f(b)$ tienen signos opuestos, así que solo 6 son alcanzables (4 si $f(x) \neq 0$). Número de generaciones (experimentos.py generaciones, no está en la guía de 90 min): con 100 el éxito es 15 % y no hay demostración completa; con 300, 74 % y la política voraz ya produce la mínima aunque durante el entrenamiento no salió ninguna (ϵ todavía alto); con 1000 ya está todo.

4 Trampas de la sesión

- Si editan `proof_kb.py` y olvidan revertir, la siguiente sesión arranca con el grafo alterado. Insistir en `git checkout bisectrl/proof_kb.py` (o en volver a descomprimir el ZIP).
- Si un distractor nuevo tiene deps no vacías o `distractor=False`, cambia `REQUIRED_KEYS`. Solo hace falta `Step("D_MIO", "Distractor", "...", distractor=True)`.
- `keep_policy_is_bolzano()` devuelve True con solo 4 estados visitados y coherentes; con 100 generaciones puede dar True aunque el éxito sea 15 %. No es contradicción: la regla ya es correcta pero el corte todavía es malo. Buena pregunta extra.
- Terminal pequeña: rich recorta paneles y parece que «falta» información. Usar `--no-tui`.

5 Qué se evalúa

Rúbrica sugerida (10 puntos): haber corrido el agente y entregado la tabla de hitos (2), (a)-(e) (2), (f)-(i) (3), (j)-(n) (3). Pesar (f) y (n): son las dos ideas de la sesión. El distractor propio es opcional y suma hasta 1 punto extra.